

PROGRAMAÇÃO ORIENTADA A OBJETOS

Aula 12 – Polimorfismo

Análise e Desenvolvimento de Sistemas

Prof.: Cícero Santos

Agenda de Aula

- **Polimorfismo P00**
 - Entender o que é polimorfismo
 - Vinculo dinâmico (Dynamic Binding)
 - Superclasse abstrata
 - Sobrescrita (Overriding) vs. Sobrecarga

Polimorfismo



- **POLI** = muitas/diversas
- **MORFO** = formas/formatos

Polimorfismo

- **Conceito**

- De acordo com Diatel (2010), o polimorfismo permite que escrevamos programas que processam objetos que compartilham a mesma superclasse como se todos fossem objetos dela

```
for(Animal a : animais) {  
    a.amitirSom();  
}
```

- Resultado: cada objeto “sabe fazer a coisa certa” em tempo de execução.

Polimorfismo



```
p = Pessoa(); ✗  
a = Aluno();  
m = Medico();  
pro = Professor();  
  
a.setNome("Pedro");  
m.setNome("Ana Leticia");  
pro.setNome("Bruno");  
  
a.trabalgar(); // estagiar  
m.trabalhar(); // atender paciente  
pro.trabalhar(); // dar aulas
```

Polimorfismo



```
p = ControleRemoto();  
a = Radio();  
m = DvdPlayer();  
  
a.tocar();    // play na música  
m.tocar();    // play no vídeo/filme
```

Polimorfismo

- **Assinatura do método**

- Nome do método
- Quantidade de parâmetros e
- Tipos dos parâmetros

**** no exemplo ao lado, os métodos tem a mesma assinatura?**

- O compilador escolhe qual executar com base nos dados.

```
public float metodo1(float a, float b) {  
    return a * b;  
}
```

```
public int metodo1(int a, int b) {  
    return a + b;  
}
```

```
public int metodo1(int a, int b, String str) {  
    return a + b;  
}
```

Polimorfismo

- **Assinatura do método**

- Nome do método
- Quantidade de parâmetros e
- Tipos dos parâmetros

** no exemplo ao lado, os métodos tem a mesma assinatura?

Não. Não tem a mesma assinatura.

```
public float metodo1(float a, float b) {  
    return a * b;  
}
```

```
public int metodo1(int a, int b) {  
    return a + b;  
}
```

```
public int metodo1(int a, int b, String str) {  
    return a + b;  
}
```


Polimorfismo

- **No conceito de polimorfismo ah dois tipos:**
 - **Sobreposição (mesma assinatura)**
 - Ela ocorre quando uma subclasse fornece uma implementação específica para um método que já foi definido em sua superclasse.
 - Acontece quando substituímos um método de uma superclasse na subclasse, usando a mesma assinatura.

```
public void reagir(String texto){  
    // sobreposição  
}  
  
public void reagir(String texto){  
    // sobreposição  
}
```

Polimorfismo

- **No conceito de polimorfismo há dois tipos:**
 - **Sobrecarga (assinaturas diferentes)**
 - Ela permite que existam vários métodos com o mesmo nome na mesma classe, mas com comportamentos diferentes baseados nos dados fornecidos.

```
public void reagir(String texto){ }  
  
public void reagir(int h, int m){ }  
  
public void reagir(boolean b){ }  
  
public void reagir(int n, double d){ }
```

Polimorfismo



- **No conceito de polimorfismo há dois tipos:**
 - **Sobreposição**
 - **Objetivo:** Permitir que um objeto se comporte de maneira específica para sua classe, mesmo quando referenciado por um tipo mais genérico (a superclasse).
 - **Sobrecarga**
 - **Objetivo:** Oferecer conveniência ao programador, permitindo que a mesma operação seja executada com diferentes tipos de entrada.

Polimorfismo

- **No conceito de polimorfismo há dois tipos:**
 - **Sobreposição**
 - **Exemplo:** Se temos uma classe `Animal` com o método `emitirSom()`, as subclasses `Cachorro` e `Gato` irão **sobrescrever** esse método para latir ou miar, respectivamente.
 - **Sobrecarga**
 - **Exemplo:** Um método `somar(int a, int b)` e outro `somar(double a, double b)`. O compilador decide qual usar com base nos argumentos passados.

Polimorfismo: Pessoa



Médico



Professor



Aluno

Polimorfismo: Pessoa

- **Classe: Pessoa**



```
abstract class Pessoa {  
    protected String nome, sobrenome;  
    protected int idade;  
    protected int tipoDocumento;  
    protegido double salario;  
  
    public abstract void trabalhar();  
    public abstract void abstract praticaEsporte();  
}
```

Polimorfismo: Pessoa

- **Classe: Medico**



```
class Medico extends Pessoa {  
    private String CRM;  
  
    @Override  
    public void trabalhar(){  
        // atender paciente  
    }  
  
    @Override  
    public void praticaEsporte(){  
        // caminhada  
    }  
}
```

Polimorfismo: Pessoa

- **Classe: Professor**



```
class Professor extends Pessoa {  
    private String Sei;  
  
    @Override  
    public void trabalhar(){  
        // dar aulas  
    }  
  
    @Override  
    public void praticaEsporte(){  
        // caminhada  
    }  
}
```


Polimorfismo: Pessoa

- **Classe: Aluno**



```
class Aluno extends Pessoa {  
    private String CRM;  
  
    @Override  
    public void trabalhar(){  
        // estagiar  
    }  
  
    @Override  
    public void praticaEsporte(){  
        // futebol  
    }  
}
```

Bora praticar...

Se liga nas instruções do professor.

Tarefa

- **Crie uma classes com as características e comportamento descritos nos slides a seguir, considere adicionar mais características e comportamento a classe, a seu critério.**

Animal: abstrata

- **Animal**
 - Peso: real
 - Idade: inteiro
 - Membros: inteiro
 - Comportamento:
 - Locomover: sem retorno
 - Alimentar: sem retorno
 - amitarSom: sem retorno

Mamífero: Animal

- **Mamífero:**
 - corPelo: texto
 - Comportamento:
 - Implemente os comportamentos

Reptil: Animal



- **Reptil:**
 - corEscama: texto
 - Comportamento:
 - Implemente os comportamentos

Peixe: Animal



- **Peixe:**
 - corEscama: texto
 - Comportamento:
 - Implemente os comportamentos

Ave: Animal

- **Ave:**
 - corEscama: texto
 - Comportamento:
 - fazerNinho: sem retorno
 - Implemente os comportamentos

Referência



DEITEL, Paul; DEITEL, Harvey. Java: como programar. 10. ed. São Paulo: Pearson Education do Brasil, 2016.

CARVALHO, Thiago Leite. Orientação a Objetos: aprenda seus conceitos e suas aplicabilidades de forma efetiva. 1. ed. São Paulo: Casa do Código, 2016.

Obrigado.

Aula 12 – Polimorfismo

Análise e Desenvolvimento de Sistemas

Prof.: Cícero Santos

